C PROGRAMMING & DATA STRUCTURES

DIGITAL MARKETING FUNNEL TREE

A Comprehensive Management System Implementing CRUD Operations on Hierarchical Marketing Stages

ARCHITECTURE	FUNCTIONALITY	ENVIRONMENT
Tree Data Structure	***CRUD Operations***	***C Language***

Project Objectives & Goals

Defining the core mission and learning outcomes of the Digital Marketing Funnel Tree project.

Master Tree Data Structures

Deep dive into hierarchical data organization, specifically focusing on non-linear structures to represent complex marketing stages.

- Dynamic Memory Allocation
- Pointer Management

Implement CRUD Operations

Develop a robust backend logic in C to Create, Read, Update, and Delete nodes within the marketing funnel tree.

- Recursive Search Algorithms
- Node Modification Logic

Simulate Marketing Funnels

Bridge the gap between data structures and real-world applications by modeling Awareness, Interest, and Conversion stages.

- Business Logic Mapping
- Hierarchical Visualization

TECHNICAL STACK

CORE TECHNOLOGIES

Project Implementation Layer

C Programming

The core engine of the system, chosen for its low-level control and efficiency in handling data structures.

- Procedural Logic
- High Performance

Tree Structures

A hierarchical model used to map the marketing funnel stages from Awareness down to Conversion.

- Parent-Child Nodes
- Recursive Traversal

Dynamic Memory

Utilizing heap memory management to allow the marketing funnel to scale dynamically at runtime.

- malloc() / free()
- Pointer Arithmetic

CRUD OPERATIONS · PART 1

ADD & DISPLAY

Funnel Construction & Visualization

Add Stage

New marketing stages are inserted as nodes into the tree. The system prompts for a `stage name` and a `parent node`, then allocates memory and links the child dynamically.

```
// ADDNODE() FLOW

1. Prompt user for stageName
2. Locate parentNode by search
3. malloc() new TreeNode struct
4. Assign name & set children = NULL
5. Link to parent's child pointer
```

● Dynamic Allocation ● Parent Linking

Display Funnel

The entire funnel hierarchy is visualised via a `recursive pre-order traversal`. Each level is indented, giving users a clear top-down map of all marketing stages.

```
// SAMPLE OUTPUT

📦 Awareness
    └ Interest
        └ Consideration
            └ Conversion
```

● Recursive Traversal ● Indented Levels

CRUD OPERATIONS · PART 2

UPDATE & SEARCH

Stage Editing & Node Discovery

Update Stage

Existing stage names are modified without restructuring the tree. The system locates the target node by name, then overwrites the `name` field in-place, preserving all child relationships.

```
// RENAMENODE() FLOW
```

01 Traverse tree to find node by current name

02 Prompt user for the new stage name

03 `strcpy(node→name, newName)`

04 Children & structure remain untouched

● In-place Edit ● Zero Restructure

Search Node

Users can locate any stage by name using a `depth-first search` (DFS). The algorithm recursively visits each node, returning the matching node pointer or `NULL` if not found.

```
// SEARCHNODE() DFS
if (node == NULL) return NULL;
if (strcmp(node→name, target) == 0)
  return node; // found!
for each child in node→children:
  result = searchNode(child, target);
  if (result) return result;
```

O(n) Time

O(h) Space (stack)

● Depth-First Search ● Recursive Logic

FEATURE HIGHLIGHT

DELETING STAGES

Safe Node Removal & Tree Integrity

Deletion Process

1. User inputs the target stage name
2. DFS locates the node & its parent pointer
3. Recursively freeSubtree() all descendants
4. Unlink node from parent's children list
5. free(node) — heap memory released

// FREESUBTREE() — C CODE

```c
void freeSubtree(Node *node) {
  if (node == NULL) return;
  Node *child = node->firstChild;
  while (child != NULL) {
    Node *next = child->nextSibling;
    freeSubtree(child);
    child = next;
  }
  free(node);
}
```

BEFORE / AFTER DELETION

BEFORE	AFTER
Awareness	Awareness
Interest	Interest
~~Consideration~~	— empty —
~~Conversion~~	

INTEGRITY GUARANTEE

- No dangling pointers
- Sibling chain re-linked
- All subtree memory freed